arXiv:2410.05355v1 [cs.CL] 7 Oct 2024

Falcon Mamba: The First Competitive Attention-free 7B Language Model

Jingwei Zuo* **Maksim Velikanov** **Dhia Eddine Rhaiem** **Ilyas Chahed**

Younes Belkada **Guillaume Kunsch** **Hakim Hacid**

Technology Innovation Institute, Abu Dhabi, United Arab Emirates

*Falcon-LLM[at]tii[dot]ae

Abstract

In this technical report, we present Falcon Mamba 7B, a new base large language model based on the novel Mamba architecture. Falcon Mamba 7B is trained on 5.8 trillion tokens with carefully selected data mixtures. As a pure Mamba-based model, Falcon Mamba 7B surpasses leading open-weight models based on Transformers, such as Mistral 7B, Llama3.1 8B, and Falcon2 11B. It is on par with Gemma 7B and outperforms models with different architecture designs, such as RecurrentGemma 9B, and RWKV-v6 Finch 7B/14B. Currently, Falcon Mamba 7B is the best-performing Mamba model in the literature at this scale, surpassing both existing Mamba and hybrid Mamba-Transformer models, according to Open LLM Leaderboard (Fourrier et al., 2024). Due to its architecture, Falcon Mamba 7B is significantly faster at inference and requires substantially less memory for long sequence generation. Despite recent studies suggesting that hybrid Mamba-Transformer models outperform pure architecture designs, we demonstrate that even the pure Mamba design can achieve similar, even superior results compared to the Transformer and hybrid designs. We make the weights of our implementation of Falcon Mamba 7B publicly available on https://huggingface.co/tiiuae/falcon-mamba-7b, under a permissive license[1].

1 Introduction

Modern foundation models are predominantly based on the Transformer and its core attention layer (Vaswani et al., 2017). Due to its quadratic complexity regarding the window length, recent research attempts to propose more efficient alternatives of vanilla attention, such as FlashAttention (Dao et al., 2022; Dao, 2024), sliding window attention (Beltagy et al., 2020). New architectures beyond Transformers such as Griffin (De et al., 2024), RWKV (Peng et al., 2023), and Mamba (Gu & Dao, 2023) have recently been proposed and have demonstrated performance comparable to Transformers. However, most of them either proved their performance at small scale, or still show a performance gap with recent Transformer-based performing LLMs.

There have been efforts from the community to scale up Mamba LLMs beyond the original test-purpose 2.8B Mamba model (Gu & Dao, 2023). Notable examples include Mamba-7B-rw (Mercat et al., 2024), Zamba 7B (Glorioso et al., 2024), Samba 3.8B (Ren et al., 2024), Mamba2 8B

[1] https://falconllm.tii.ae/falcon-mamba-7b-terms-and-conditions.html

(hybrid/non-hybrid) (Waleffe et al., 2024). Most of these models adopt a hybrid Mamba-Transformer design, demonstrating superior performance compared to pure Transformer models. However, it remains unclear whether a pure attention-free model can match the performance of highly optimized Transformers at large data and model size scales.

We introduce Falcon Mamba 7B — a base (pre-trained) model with pure mamba architecture design, and the first State Space Language Model (SSLM) in the FalconLLM series. We argue that Falcon Mamba 7B answers the above question positively, and, to the best of our knowledge, the first model to do so. As measured by Open LLM Leaderboard (Fourrier et al., 2024) collection of benchmarks, Falcon Mamba 7B matches or surpass powerful transformer-based pretrained LLMs such as Llama3.1 8B (Dubey et al., 2024), Mistral 7B (Jiang et al., 2023) and Falcon2 11B (Malartic et al., 2024). Moreover, it outperforms models with other architectural designs, such as RecurrentGemma 9B (Botev et al., 2024) based on Griffin and RWKV-v6 Finch 7B and 14B (Peng et al., 2024). More importantly, with the pure Mamba design, Falcon Mamba 7B maintains constant memory cost regardless of the context length, while providing extreme efficient inference for extreme long context data generation.

In this technical report, we provide a detailed overview of the model architecture, training recipes and pretraining data preparations for Falcon Mamba 7B. This will be followed by detailed comparisons with LLMs with different architecture designs on popular benchmarks. Finally, we show the broader implications of Falcon Mamba 7B, its limitations and advantages, and conclusions.

2 Model Architecture

The Falcon Mamba 7B model architecture is based on Mamba (Gu & Dao, 2023). The core parameters of the architectures are summarized in Table 1.

Table 1: Model Parameters of Falcon Mamba 7B

Params	n_layers	d_model	exp. factor E	vocab_size	tied_embedding	d_conv	Δ proj. size	state dim. (N)
7.27B	64	4096	2	65024	False	4	16	16

We have untied the input embeddings from the output weights throughout the entire training process to increase model flexibility. Based on our experimental results, this approach has led to improved model performance at the 7B scale.

Note that, in contrast to transformers, the sequence length is not a part of Mamba architecture. Any sequence length can be used during inference, while the actual ability of the model to process long sequences is determined by the sequence length used for training.

Design decision: Recent work (Dao & Gu, 2024; Lieber et al., 2024) suggests that a hybrid architecture, with interleaved attention and SSM layers, can outperform pure Transformer or SSM models. This improvement is hypothesized to arise from the complementary features from both models: the general sequence-to-sequence mapping capabilities of SSMs and the fast retrieval properties of attention layers. Recent Mamba-based language models follows this intuition and scale up the hybrid design beyond 2.8B models, such as Samba 3.8B (Ren et al., 2024), Zamba 7B (Glorioso et al., 2024), Jamba 12B/52B (Lieber et al., 2024). However, introducing attention layers compromises the linear scalability of the Mamba architecture, prompting the question: can a purely Mamba-based design achieve competitive performance against state-of-the-art (SoTA) open LLMs at scale, while conserving its linear scalability? Recent attention-free models, such as RWKV-v6 (Peng et al., 2024), show their performance at small scale or/and on certain academic benchmarks. However, they are far behind popular LLMs when setting up more thorough comparisons on various benchmarks.

Model stability During pre-training, we observed consistent loss spikes that occurred randomly and unpredictably. Notably, when we applied higher learning rates, the model exhibited more pronounced loss spikes and became more prone to divergence. This phenomenon was also observed in the training of Falcon2 (Malartic et al., 2024), and recent papers like Jamba (Lieber et al., 2024) and Mamba2 (Dao & Gu, 2024) have reported similar issues. In particular, we found that the Mamba architecture is more sensitive to learning rates than Transformers. Careful model initializations and reducing model's learning rate sensibility are crucial for addressing this issue. Aligned with (Dehghani et al., 2023), it's becoming a common practice to apply pre-norm and post-norm with

RMSNorm in each Transformer block to stabilize the pre-training (Team et al., 2024; Yang et al., 2024). Similarly, we add RMSNorm layers after B, C and Δ. From our experiments, it appears to bring a more stable training loss than other settings, such as putting an RMSNorm layer in each block before the final output projection (Dao & Gu, 2024). This is aligned with the Jamba model designs (Lieber et al., 2024).

3 Pre-training

3.1 Training stategy

Falcon-Mamba-7B was trained on 256 H100 80GB GPUs for the majority of the training, using only Data Parallelism (DP=256). This was combined with ZeRO optimization to efficiently manage memory and training processes.

The model was trained using the AdamW optimizer with $\beta_1 = 0.9$ and $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, and weight decay value 0.1. Although we didn't apply Z-loss on output logits during Falcon-Mamba-7B pre-training, in the follow-up experiments we observed that it helps to stabilize the training, in agreement with (Wortsman et al., 2024).

We applied warmup-stable-decay (WSD) learning rate schedule (Hu et al., 2024) with a fixed warmup duration of 1GT, and learning rate $\eta_{\text{max}} = 6.4 \times 10^{-4}$ during the stable stage. This way, our model was trained with a relatively high learning rate during most of the pertaining, leading to a quick adaptation to data distribution shifts introduced between different training stages and the beginning of the decay stage (see section 3.2.2). In the decay stage, we reduced learning rate to the minimal value $\eta_{\text{min}} = \frac{\eta_{\text{max}}}{256}$ using exponential schedule with profile $\eta(t) = \eta_{\max} \exp\big[-\frac{t}{t_{\text{decay}}} \log \frac{\eta_{\max}}{\eta_{\min}}\big]$, where t_{decay} is the duration of the decay stage. Contrary to most technical reports, we found out that longer LR decay stage provided better results evaluation-wise. We kept around 10% of the total training tokens for the decay to have optimal performances, which is aligned with recent miniCPM's conclusions (Hu et al., 2024).

In the beginning of the training, we used batch size rampup. Specifically, we were linearly increasing the batch size initial value $b_{\text{min}} = 128$ to the maximum value $b_{\text{max}} = 2048$ over the first 50GT. In our experiments, we noticed that batch size rampup affects the loss curve and final model performance. This effect is most conveniently interpreted in terms of gradients *noise temperature* T_{noise}, defined for Adam optimizer as (Malladi et al., 2022)

$$T_{\text{noise}} = \frac{\eta}{\sqrt{b}}. \tag{1}$$

During batch size rampup, noise temperature (1) is decreased. This leads to better loss during the stable LR phase but a smaller loss boost within LR decay phase. To counter this deficiency, we apply *batch scaling*: keeping the Adam noise $\frac{\eta}{\sqrt{b}}$ temperature constant by adjusting learning rate η whenever batch size b is changed. We have found that batch scaling leads to a better final loss after the LR decay stage, even during long training durations much exceeding the length of rampup period.

3.2 Pre-training data

Falcon Mamba 7B was mostly trained on the data from Falcon2-11B (Malartic et al., 2024). Since a 7B model may not be sufficient to perform promising performances on multilingual tasks without harming the English ones, we exclude multilingual data from the pre-training corpus. Nevertheless, a continual pre-training stage can be adopted to empower the model with multilingual capabilities. We adopt the same tokenizer as the *Falcon* series model (Almazrouei et al., 2023) with no change.

3.2.1 Data sources

The model was trained on a diverse data mixture consisting primarily of web, curated, code, and math data.

Web data We mainly leveraged `RefinedWeb` (Penedo et al., 2023), which is a high-quality English pre-training dataset composed of five trillion tokens coming from web data only. Starting from raw Common Crawl data, samples were filtered out through language identification, filtering (line-wise and document-wise) as well as fuzzy and exact deduplication.

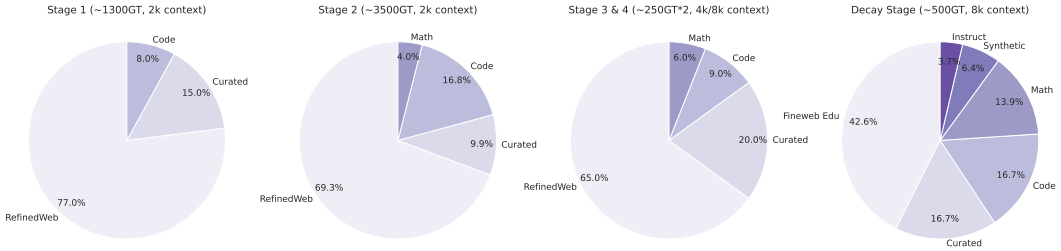

Figure 1: Data mixtures across training stages

Curated data The curated dataset includes books, scientific publications (e.g., arXiv, PubMed), patents (USPTO), and conversations from platforms like Reddit, StackExchange, and Hackernews. To properly handle conversation trees, we applied the same method as in (Malartic et al., 2024) to enforce causal temporality, ensuring that each conversation was used only once during training.

Code Samples were taken from `The Stack` (Kocetkov et al., 2022) and passed through the same processing pipelines as used for web data. Code data were gradually injected during pretraining, along with continuous data collection and processing.

Math We used `Proof-Pile-2` (Azerbayev et al., 2023) without further refinement, along with math data filtered from web using a *FastText* classifier trained on `Proof-Pile-2`.

3.2.2 Data mixtures

The pre-training was conducted in four constant learning rate (LR) stages, followed by a final LR decay stage. The first four stages consisted in progressively increasing the sequence length, from 2048 up to 8192. Following the curriculum learning concept, we carefully selected data mixtures throughout the training stages as shown in Fig. 1, considering both data diversity and complexity. The main idea is to increase high quality and scientific data at late stages. Due to limited data from certain resources, we applied multiple epochs for less-represented data, e.g., math, code, curated data. Since the packing tokens were used in the pretraining, we carefully selected the proportions of short and long samples at each stage to prevent any distribution shifts.

In the decay stage, we introduced more diverse and higher-quality data to refine or shapen the knowledge learned during earlier stages. This included using parts of Fineweb-Edu (Penedo et al., 2024) as web data, along with synthetic data from Cosmopedia (Ben Allal et al., 2024). Additionally, a small portion of multitask instruction data (four epochs for 3.7%) was used, similar to other studies (Hu et al., 2024; Yang et al., 2024), to enhance the model's zero-shot and few-shot learning capabilities. The inclusion of instruction data during pretraining is a debated topic, as it may potentially reduce a model's fine-tuning flexibility. However, from our experimental results, we found that keeping a minimal amount of instruction data enhances Mamba's in-context retrieval ability (Wen et al., 2024) while not overfitting the multitask data with limited epochs of repetitions. Additionally, we observed that the training loss was still decreasing at the end of Stage 4, suggesting that the model's performance could be further improved with continued training on more high-quality data. To support the community in further research or continual training on the model, we decided to release as well the pre-decay checkpoint [2] of the model.

4 Evaluation and Results

4.1 Benchmark results

We conducted a comparative evaluation of our model against state-of-the-art models across three distinct architectural categories: State Space Models (SSMs), Transformers, and Hybrid models. The Hybrid models integrate a combination of attention mechanisms with Recurrent/Mamba blocks.

[2] `https://huggingface.co/tiiuae/falcon-mamba-7b-pre-decay`

Benchmarks were selected where results are publicly available and independently conducted by HuggingFace, which span a broad range of top-level categories to assess the model's versatility and performance across various tasks:

- Instruction following: IFEval (0-shot) (Zhou et al., 2023)
- Math, reasoning, and problem-solving: GSM8K (5-shots) (Cobbe et al., 2021), MATH-Lvl5 (4-shots) (Hendrycks et al., 2021), ARC Challenge (25-shots) (Clark et al., 2018), GPQA (0-shot) (Rein et al., 2023), MuSR (0-shot) (Sprague et al., 2023)
- Aggregate: MMLU (5-shots) (Hendrycks et al., 2020), MMLU-Pro (5-shots) (Wang et al., 2024), BIG-Bench Hard (BBH) (3-shots) (Suzgun et al., 2022)

As shown in Table 2 and Table 3, wherever possible, we extracted results for competitor models from the HF Leaderboards v1(Beeching et al., 2023) and v2(Fourrier et al., 2024), ensuring an unbiased comparison. When leaderboard results were unavailable, we used the best available results, either from reported findings or our internal evaluations. Internal evaluations were performed using the `lm-evaluation-harness` (Gao et al., 2024) and `lighteval` (Fourrier et al., 2023) packages.

Table 2: Model Performance on HF Leaderboard v1 tasks: **bold** (best), underline (second best)

Model Name	ARC-25	HellaSwag-10	MMLU-5	Winogrande-5	TruthfulQA-0	GSM8K-5	Average
RWKV models							
RWKV-v6-Finch-7B*	43.86	75.19	41.69	68.27	42.19	19.64	48.47
RWKV-v6-Finch-14B*	47.44	78.86	52.33	71.27	45.45	38.06	55.57
Transformer models							
Falcon2-11B	59.73	82.91	58.37	78.30	52.56	53.83	**64.28**
Meta-llama-3-8B	60.24	82.23	**66.70**	78.45	42.93	45.19	62.62
Meta-llama-3.1-8B	58.53	82.13	66.43	74.35	44.29	47.92	62.28
Mistral-7B-v0.1	59.98	**83.31**	64.16	78.37	42.15	37.83	60.97
Mistral-Nemo-Base-2407 (12B)	57.94	82.82	64.43	73.72	49.14	**55.27**	63.89
Gemma-7B	61.09	82.20	64.56	79.01	44.79	50.87	63.75
Hybrid SSM-attention models							
RecurrentGemma-9b**	52.00	80.40	60.50	73.60	38.60	42.60	57.95
Zyphra/Zamba-7B-v1*	56.14	82.23	58.11	**79.87**	52.88	30.78	60.00
Pure SSM models							
TRI-ML/mamba-7b-rw*	51.25	80.85	33.41	71.11	32.08	4.70	45.52
FalconMamba-7B (pre-decay)*	49.23	80.25	57.27	70.88	37.28	21.83	57.29
FalconMamba-7B*	**62.03**	80.82	62.11	73.64	**53.42**	52.54	64.09

Table 3: Model Performance on HF Leaderboard v2: **bold** (best), underline (second best)

Model Name	IFEval-0	BBH-3	Math-Lvl5-4	GPQA-0	MuSR-0	MMLU-PRO-5	Average
RWKV models							
RWKV-v6-Finch-7B	27.65	9.04	1.11	2.81	2.25	5.85	8.12
RWKV-v6-Finch-14B	29.81	12.89	1.13	5.01	3.16	11.3	10.55
Transformer models							
Falcon2-11B	32.61	21.94	2.34	2.80	7.53	15.44	13.78
Meta-llama-3-8B	14.55	24.50	3.25	7.38	6.24	24.55	13.41
Meta-llama-3.1-8B	12.70	25.29	4.61	6.15	8.98	24.95	13.78
Mistral-7B-v0.1	23.86	22.02	2.49	5.59	10.68	22.36	14.50
Mistral-Nemo-Base-2407 (12B)	16.83	**29.37**	4.98	5.82	6.52	**27.46**	15.08
Gemma-7B	26.59	21.12	**6.42**	4.92	**10.98**	21.64	**15.28**
Hybrid SSM-attention models							
RecurrentGemma-9b	30.76	14.80	4.83	4.70	6.60	17.88	13.20
Zyphra/Zamba-7B-v1*	24.06	21.12	3.32	3.03	7.74	16.02	12.55
Pure SSM models							
TRI-ML/mamba-7b-rw*	22.46	6.71	0.45	1.12	5.51	1.69	6.25
FalconMamba-7B (pre-decay)*	24.05	11.01	1.71	3.05	8.68	8.59	9.52
FalconMamba-7B	**33.36**	19.88	3.63	**8.05**	10.86	14.47	15.04

Note: * indicates internal evaluations, ** denotes results taking from paper or model card.

Globally, Falcon-Mamba-7B outperforms models of similar scale, regardless of architecture, including Transformer models (Llama3/3.1-8B (Dubey et al., 2024), Mistral-7B (Jiang et al., 2023)), RWKV-v6-Finch-7B (Peng et al., 2024), and hybrid models like Zamba-7B, as well as Mamba-7B-RW (Mercat et al., 2024). Furthermore, it either outperforms or is comparable to larger models, such as Falcon2-11B (Malartic et al., 2024), RWKV-v6-Finch-14B (Peng et al., 2024), Gemma-7B (8.54B) (Team

et al., 2024), RecurrentGemma-9B (Botev et al., 2024), and Mistral-Nemo-12B [3]. This positions Falcon-Mamba-7B as the first competitive attention-free 7B model in the community, with promising performance across a variety of tasks. We also report the model's performance at the pre-decay checkpoint, with a notable performance boost observed during the decay stage. The decay stage can provide valuable insights for determining data mixtures in larger-scale models and simulating a condensed pretraining phase.

While recent studies (Waleffe et al., 2024; Wen et al., 2024) indicate that pure Mamba/Mamba2 designs lag behind Transformers in tasks like copying and in-context learning, Falcon-Mamba-7B has shown promising performance in few-shot learning tasks, such as MMLU, ARC-25, and GSM8K. This suggests that the quality of data and training strategies during pretraining play a more crucial role than the architecture itself and can mitigate these potential disadvantages. Moreover, Falcon-Mamba-7B excels in long-context reasoning tasks, e.g., MuSR (Sprague et al., 2023), highlighting its significant potential in long-context learning scenarios.

4.2 Throughput and memory consumption

The attention mechanism is inherently limited in processing long sequences due to the increasing compute and memory costs as sequence length grows. Leveraging the theoretical efficiency of SSM models in handling large sequences (Gu & Dao, 2023), Falcon-Mamba-7B demonstrates that these scaling limitations can indeed be overcome without compromising performance.

Setup To replicate real-world use cases, we compared the memory usage and generation throughput of Falcon-Mamba-7B with popular Transformer-based models of a similar scale, including Llama3.1-8B (Dubey et al., 2024), Mistral-7B (Jiang et al., 2023), and Qwen2-7B (Yang et al., 2024). All evaluations were conducted using the Hugging Face `transformers` library (Wolf et al., 2020). For a fair comparison, we rescaled the vocabulary size of all transformer models to match Falcon-Mamba-7B, since it has a big impact on the memory footprint of the model.

Parallel Prefill and Sequential Prefill Before diving into the results, it is important to clarify the difference between the prompt (prefill) and generated (decode) parts of a sequence. For state space models (SSMs), the prefill process is more critical than for transformer models. When a transformer generates the next token, it must attend to the keys and values of all previous tokens in the context, resulting in both memory and generation time scaling linearly with context length. In contrast, an SSM only stores and attends to its recurrent state, which avoids the need for additional memory or time when generating large sequences. While this demonstrates the efficiency of SSMs during the decoding phase, the prefill phase requires additional framework optimizations to fully leverage the SSM architecture.

The standard method for prefill is processing the entire prompt in parallel, maximizing GPU utilization, referred to here as **Parallel Prefill**. This is the approach used in most frameworks like `Optimum-Benchmark` [4]. In this approach, the memory usage grows with prompt length due to the need to store hidden states for each token. For transformers, memory is dominated by stored key-value (KV) caches, whereas SSMs don't require KV caching. However, for SSMs, the memory required to store hidden states still scales with the prompt length, making it challenging to handle arbitrarily long sequences, similar to transformers. An alternative method, which we referred to as **Sequential Prefill**, processes the prompt token by token (or in larger chunks for better GPU usage), similar to sequence parallelism. While this method offers little benefit for transformers, it allows SSMs to process arbitrarily long prompts, mitigating the memory scaling issue seen with parallel prefill. This requires more community supports for optimizing existing inference frameworks for SSMs.

With these considerations in mind, we first evaluate the maximum sequence length that can fit on a single 24 GB A10 GPU, as shown in Fig. 2. The batch size is fixed at 1, and we employ float32 precision for all operations. Our results show that, even for parallel prefill, Falcon-Mamba-7B is capable of fitting larger sequences compared to a standard transformer architecture, while in sequential prefill, Falcon-Mamba-7B can unlock its full potential and process arbitrarily long prompts.

[3] `https://huggingface.co/mistralai/Mistral-Nemo-Base-2407`

[4] `https://github.com/huggingface/optimum-benchmark`

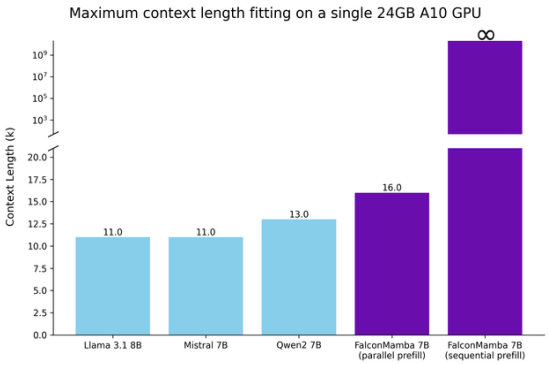

Figure 2: We vary the context length of the prompt to determine the maximum sequence length that could be processed without encountering an out-of-memory (OOM) error. To ensure a fair comparison, all models were configured with a rescaled vocabulary size.

Next, we evaluate the generation throughput in an extreme setting: a prompt of length 1 and up to 130k generated tokens, using a batch size of 1 on an 80GB H100 GPU. The results, reported in Fig. 3, reveal that Falcon-Mamba-7B maintains a constant throughput across all generated tokens, without any increase in peak CUDA memory usage. In contrast, the Mistral-7B model exhibits a linear increase in peak memory consumption, and its generation speed decreases as the number of generated tokens grows.

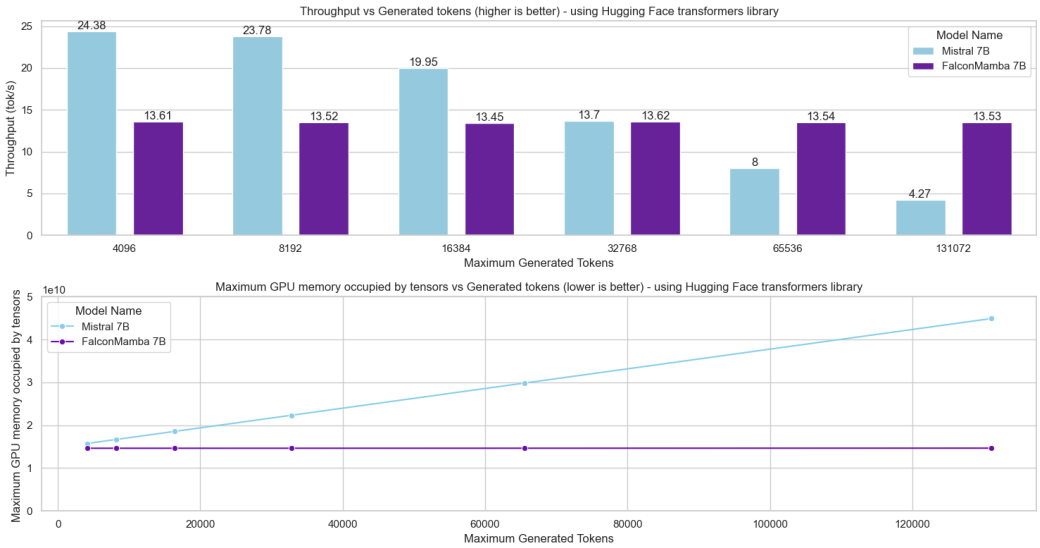

Figure 3: With a fixed batch size and context length of 1, we vary the generated tokens up to 130k for Faclon-Mamba-7B, and Mistral-7B with a resized vocabulary for fair comparisons.

5 Model Integration and Availability

5.1 Batched generation support

In real-world scenarios, input sequences of varying lengths are often batched together for efficiency, which introduces padding tokens to align the sequences. This can pose challenges for SSM-based models like Mamba, as right-side padding, while effective during training—where padding tokens are masked out in the loss computation—becomes problematic during inference. In inference, the Mamba model predicts the next token based on all previous hidden states, so including padding tokens from shorter sequences can lead to inaccurate predictions.

In Transformer models, left-side padding is typically used to prevent padding tokens from interfering with the attention mechanism. For Mamba models, which use both SSMs and convolutional layers, a different approach is required. Apart from left-side padding, Falcon-Mamba-7B handles this by zeroing out the hidden states for left padding tokens both before and after the causal convolution step. This ensures padding tokens do not influence the model's predictions during generation.

5.2 Model Availability

The Falcon-Mamba-7B models, including the pre-decay checkpoint, are made available under the Falcon Mamba 7B TII License [5], a permissive Apache 2.0-based software license which includes an acceptable use policy [6] that promotes the responsible use of AI.

The models are fully integrated within the Hugging Face ecosystem and can be accessed through the Transformers library (Wolf et al., 2020). This includes support for inference, quantization (using most supported quantization schemes), and fine-tuning via the TRL library (von Werra et al., 2020). All associated artifacts, including GGUF files, can be browsed through the Falcon Mamba 7B collection in Hugging Face.

Additionally, support for Falcon-Mamba-7B has been added to the `llama.cpp` package [7], enabling easy deployment of Falcon-Mamba-7B on local machines using CPU hardware. We are planning to expand the support for more platforms in the future.

6 Discussions and conclusion

We have introduced Falcon Mamba 7B, the first competitive 7B language model based purely on the Mamba architecture. Our results show that it matches or outperforms state-of-the-art transformer models such as Llama 3.1 and Mistral 7B in a variety of benchmarks. This way, Falcon Mamba 7B sets a new benchmark for attention-free models, proving that pure SSM-based designs can achieve state-of-the-art performance. We hope that our model will strengthen the belief in further innovation of efficient language model architectures, challenging the infamous "attention is all you need" saying.

The main advantage of mamba architecture lies in the long-context generation, where it maintains constant memory and throughput usage regardless of sequence length. We have confirmed this statement with throughput and memory analysis for Falcon Mamba 7B. However, as we focused on obtaining a strong general-purpose language model, the actual proficiency of the model in long sequence understanding and generation was not emphasized in Falcon Mamba 7B training strategy, featuring rather medium 8k context length. Tailoring the training procedure towards extra-large contexts and verifying mamba proficiency in this regime remains an important yet underexplored area for future research and development. If successful, it would make mamba-based models ideal for real-world applications requiring low-latency, large-scale generation, e.g., audio, video.

While Falcon Mamba 7B performs well, particularly in reasoning tasks and long-context learning, it shows potentially some limitations in in-context learning compared to Transformers. Although high-quality data, especially Chain-of-Thought (CoT) instruction data or tailored prompting techniques (Arora et al., 2024), help mitigate these potential disadvantages, it may still not be sufficient to close the gap with Transformers (Wen et al., 2024), given the same data budget. However, data scaling and model scaling in the Mamba architecture have been less explored in the literature, leaving the potential limitations and optimizations of Mamba as an open area for further research. Moreover, the complementary features of sequence mixing performed by SSM and attention suggest that hybrid models might have the best of both worlds. Although many recent models (Lieber et al., 2024; Ren et al., 2024; Dao & Gu, 2024; De et al., 2024) have started to explore this direction, we believe that the question of how to optimally use SSM and attention in a single architecture remains open.

[5] `https://falconllm.tii.ae/falcon-mamba-7b-terms-and-conditions.html`

[6] `https://falconllm.tii.ae/falcon-mamba-7b-acceptable-use-policy.html`

[7] `https://github.com/ggerganov/llama.cpp`

Acknowledgments

We would like to thank the Hugging Face team for their continuous support and model integration within their ecosystem. We also extend our gratitude to Tri Dao and Albert Gu for implementing and open-sourcing the Mamba architecture for the community.

References

Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, et al. The falcon series of open language models. *arXiv preprint arXiv:2311.16867*, 2023.

Simran Arora, Aman Timalsina, Aaryan Singhal, Benjamin Spector, Sabri Eyuboglu, Xinyi Zhao, Ashish Rao, Atri Rudra, and Christopher Ré. Just read twice: closing the recall gap for recurrent language models. *arXiv preprint arXiv:2407.05483*, 2024.

Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. *arXiv preprint arXiv:2310.10631*, 2023.

Edward Beeching, Clémentine Fourrier, Nathan Habib, Sheon Han, Nathan Lambert, Nazneen Rajani, Omar Sanseviero, Lewis Tunstall, and Thomas Wolf. Open llm leaderboard (2023-2024). `https://huggingface.co/spaces/open-llm-leaderboard-old/open_llm_leaderboard`, 2023.

Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Loubna Ben Allal, Anton Lozhkov, Guilherme Penedo, Thomas Wolf, and Leandro von Werra. Cosmopedia, 2024. URL `https://huggingface.co/datasets/HuggingFaceTB/cosmopedia`.

Aleksandar Botev, Soham De, Samuel L Smith, Anushan Fernando, George-Cristian Muraru, Ruba Haroun, Leonard Berrada, Razvan Pascanu, Pier Giuseppe Sessa, Robert Dadashi, et al. Recurrentgemma: Moving past transformers for efficient open language models. *arXiv preprint arXiv:2404.07839*, 2024.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Soham De, Samuel L Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, et al. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv preprint arXiv:2402.19427*, 2024.

Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. Scaling vision transformers to 22 billion parameters. In *International Conference on Machine Learning*, pp. 7480–7512. PMLR, 2023.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Clémentine Fourrier, Nathan Habib, Thomas Wolf, and Lewis Tunstall. Lighteval: A lightweight framework for llm evaluation, 2023. URL `https://github.com/huggingface/lighteval`.

Clémentine Fourrier, Nathan Habib, Alina Lozovskaya, Konrad Szafer, and Thomas Wolf. Open llm leaderboard v2. `https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard`, 2024.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 07 2024. URL `https://zenodo.org/records/12608602`.

Paolo Glorioso, Quentin Anthony, Yury Tokpanov, James Whittington, Jonathan Pilault, Adam Ibrahim, and Beren Millidge. Zamba: A compact 7b ssm hybrid model. *arXiv preprint arXiv:2405.16712*, 2024.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, et al. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.

Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, et al. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

Quentin Malartic, Nilabhra Roy Chowdhury, Ruxandra Cojocaru, Mugariya Farooq, Giulia Campesan, Yasser Abdelaziz Dahou Djilali, Sanath Narayan, Ankit Singh, Maksim Velikanov, Basma El Amel Boussaha, et al. Falcon2-11b technical report. *arXiv preprint arXiv:2407.14885*, 2024.

Sadhika Malladi, Kaifeng Lyu, Abhishek Panigrahi, and Sanjeev Arora. On the SDEs and scaling rules for adaptive gradient algorithms. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho (eds.), *Advances in Neural Information Processing Systems*, 2022. URL `https://openreview.net/forum?id=F2mhzjHkQP`.

Jean Mercat, Igor Vasiljevic, Sedrick Keh, Kushal Arora, Achal Dave, Adrien Gaidon, and Thomas Kollar. Linearizing large language models. *arXiv preprint arXiv:2405.06640*, 2024.

Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. The refinedweb dataset for falcon llm: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116*, 2023.

Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, Thomas Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, et al. Rwkv: Reinventing rnns for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

Bo Peng, Daniel Goldstein, Quentin Anthony, Alon Albalak, Eric Alcaide, Stella Biderman, Eugene Cheah, Teddy Ferdinan, Haowen Hou, Przemysław Kazienko, et al. Eagle and finch: Rwkv with matrix-valued states and dynamic recurrence. *arXiv preprint arXiv:2404.05892*, 2024.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.

Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. Samba: Simple hybrid state space models for efficient unlimited context language modeling. *arXiv preprint arXiv:2406.07522*, 2024.

Zayne Sprague, Xi Ye, Kaj Bostrom, Swarat Chaudhuri, and Greg Durrett. Musr: Testing the limits of chain-of-thought with multistep soft reasoning. *arXiv preprint arXiv:2310.16049*, 2023.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.

Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. Trl: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2020.

Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, et al. An empirical study of mamba-based language models. *arXiv preprint arXiv:2406.07887*, 2024.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.

Kaiyue Wen, Xingyu Dang, and Kaifeng Lyu. Rnns are not transformers (yet): The key bottleneck on in-context retrieval. *arXiv preprint arXiv:2402.18510*, 2024.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Perric Cistac, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-Art Natural Language Processing. pp. 38–45. Association for Computational Linguistics, October 2020. URL `https://www.aclweb.org/anthology/2020.emnlp-demos.6`.

Mitchell Wortsman, Peter J Liu, Lechao Xiao, Katie E Everett, Alexander A Alemi, Ben Adlam, John D Co-Reyes, Izzeddin Gur, Abhishek Kumar, Roman Novak, Jeffrey Pennington, Jascha Sohl-Dickstein, Kelvin Xu, Jaehoon Lee, Justin Gilmer, and Simon Kornblith. Small-scale proxies for large-scale transformer training instabilities. In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=d8w0pmvXbZ`.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.